Instituto Superior de Engenharia de Lisboa
Departamento de Engenharia Eletrónica e Telecomunicações e de Computadores (DEETC)
Mestrado em Engenharia Informática e de Computadores

Módulo de Prática #06

Avaliação de Modelos

Aprendizagem e Mineração de Dados (AMD)

1º Semestre – 2025/2026



Autores:

Florin Sofinet (A46608)
Diogo Guerra (A49149)
Nuno Silva (A53976)

4 de dezembro de 2025

Conteúdo

1 Introdução **3**

2 Orange Data Mining **3**
2.1 Objetivo . . . 3
2.2 1.a) Instalação da biblioteca pandas . . . 3
2.3 1.b) Verificação das versões das bibliotecas . . . 3

3 Carregamento de um Dataset **4**
3.1 Objetivo . . . 4
3.2 2.a) Execução do ficheiro original . . . 4
3.3 2.b) Carregamento do dataset Pima Indians Diabetes . . . 4
3.4 2.c) Exploração da função load_dataset . . . 5
3.5 2.d) Retorno ao simple_dataset . . . 5

4 Aplicação de Métodos de Separação Train-Test **6**
4.1 Objetivo . . . 6
4.2 3.a) Método Holdout . . . 6
4.3 3.b) Método Holdout com Divisão 50/50 . . . 7
4.4 3.c) Método Stratified Holdout . . . 8
4.5 3.d) Método Repeated Stratified Holdout . . . 8

5 Código Completo de Train-Test e Análise de Resultados **9**
5.1 Objetivo . . . 9
5.2 4.a) Implementação de Métodos Adicionais . . . 9
5.3 4.b) Análise de Estratégias . . . 10
5.4 4.c) Resultados e Incompatibilidades . . . 10

6 Análise e Implementação Bootstrap **10**
6.1 Objetivo . . . 10
6.2 5.a) Execução do Script Bootstrap . . . 11
6.3 5.b) Correção do Conjunto de Teste . . . 11
6.4 5.c) Teste com Diferentes Repetições . . . 11
6.5 5.d) Integração no Script Principal . . . 12

7 Construir um Classificador com Divisão Treino-Teste **12**
7.1 Exercício 6 (b): Aplicar o DecisionTreeClassifier . . . 12
7.1.1 Implementação . . . 13
7.2 Exercício 6 (c): Aplicar a Divisão Holdout . . . 13
7.2.1 Implementação . . . 13
7.2.2 Método de Divisão Holdout . . . 13
7.2.3 Implementação . . . 14
7.2.4 Classificador: Árvore de Decisão . . . 14
7.2.5 Conjunto de Dados: Pima Indians Diabetes . . . 14
7.2.6 Resultados . . . 15

8 Múltiplas Métricas e Divisões Repetidas **15**
8.1 (d) Completar a Função score_recipe 15
8.2 (e) Testar o Código 15
8.3 (f) Observar os Avisos 16
8.4 (g) Explicar a Diferença entre Avisos 16
8.4.1 Diferença entre Avisos 16
8.4.2 Aviso: no true samples (recall e f1-score) 17
8.4.3 Aviso: no predicted samples (precision e f1-score) 17
8.5 (h) Utilizar o Parâmetro zero_division 17
8.6 (i) Desativar Avisos UndefinedMetricWarning 18

9 Conclusão **18**
9.1 Competências Adquiridas 18
9.2 Principais Aprendizagens 19
9.3 Visão Final 19

1 Introdução

Este laboratório tem como objetivo explorar métodos de avaliação de modelos de Machine Learning, combinando capacidades programáticas de Python com ferramentas de outras bibliotecas e do ambiente do Orange Data Mining.

Ao longo do trabalho, serão analisadas diversas técnicas de divisão de dados, métodos de amostragem, bootstrap, métricas de desempenho e construção de classificadores.

Os objetivos principais são:

1. Integrar bibliotecas Python no ambiente Orange Data Mining
2. Carregar e processar datasets para análise
3. Compreender e implementar métodos de divisão treino-teste (holdout, stratified holdout, holdout repetido)
4. Implementar e analisar métodos avançados de amostragem (leave-one-out, leave-p-out, k-fold, bootstrap)
5. Construir classificadores utilizando scikit-learn
6. Implementar múltiplas métricas de avaliação (exatidão, precisão, revocação, F1-score, Kappa de Cohen)
7. Analisar avisos de métricas e compreender as suas causas
8. Comparar desempenho em conjuntos de dados de diferentes tamanhos

2 Orange Data Mining

2.1 Objetivo

Integrar a biblioteca pandas no ambiente virtual do Orange Data Mining para preparar o ambiente de trabalho necessário para os exercícios seguintes.

2.2 1.a) Instalação da biblioteca pandas

Foi instalada a biblioteca pandas no ambiente virtual do Orange utilizando o gestor de pacotes pip. Esta biblioteca é essencial para manipulação e análise de dados tabulares.

2.3 1.b) Verificação das versões das bibliotecas

Verificamos as versões das bibliotecas instaladas no ambiente virtual:

- **pandas:** Biblioteca para manipulação de dados
- **numpy:** Biblioteca para computação numérica
- **scikit-learn:** Biblioteca para machine learning

Esta verificação garante que todas as dependências necessárias estão corretamente instaladas e com versões compatíveis para os exercícios seguintes.

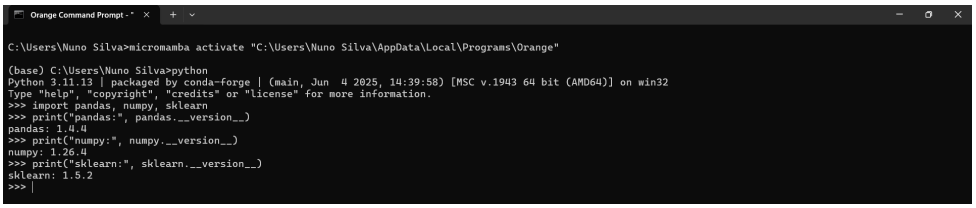


Figura 1: Verificação das versões de pandas, numpy e scikit-learn instaladas no ambiente virtual

3 Carregamento de um Dataset

3.1 Objetivo

Compreender como os datasets são lidos, processados e disponibilizados para análise através de diferentes métodos de carregamento.

3.2 2.a) Execução do ficheiro original

Executamos o ficheiro `my_split_and_eval.py` original sem alterações para compreender o comportamento padrão do script e verificar o seu correto funcionamento.

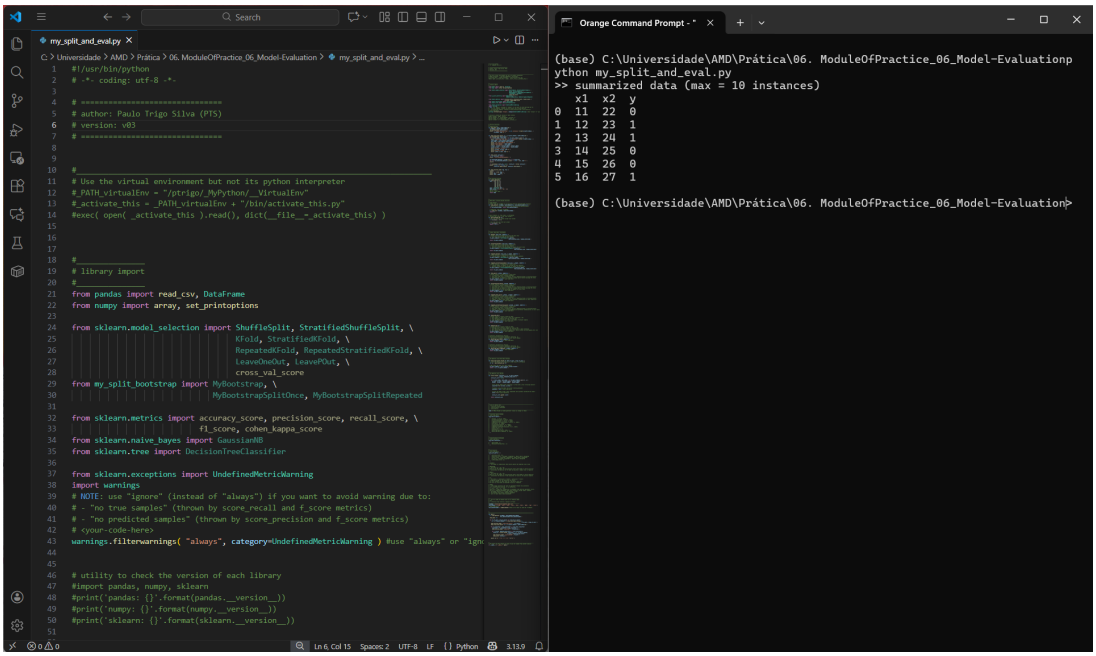


Figura 2: Execução do ficheiro my_split_and_eval.py sem alterações

3.3 2.b) Carregamento do dataset Pima Indians Diabetes

Substituímos a função de carregamento do `simple_dataset` pelo carregamento direto do ficheiro `pima-indians-diabetes.data.csv`.

Este dataset contém:

- **Instâncias:** 768 pacientes
- **Características:** 8 características médicas (gravidez, glucose, pressão arterial, espessura da pele, insulina, IMC, pedigree de diabetes, idade)
- **Classes:** 2 (diabético vs. não-diabético)

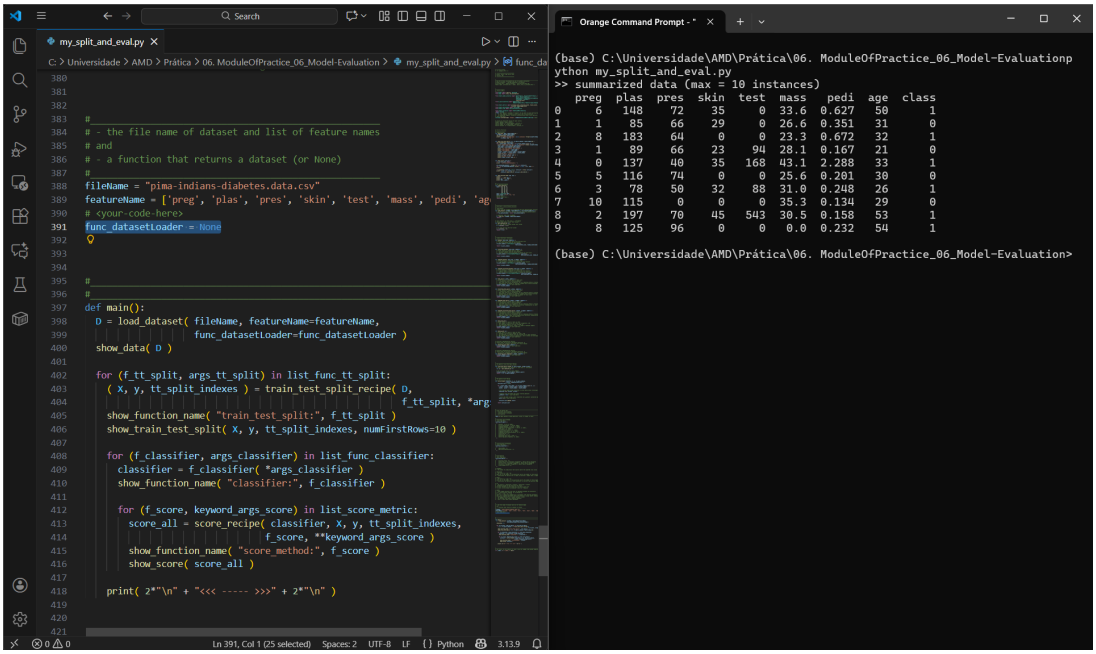


Figura 3: Substituição da função simple_dataset pelo carregamento direto do dataset Pima

3.4 2.c) Exploração da função load_dataset

Exploramos o funcionamento da função `load_dataset` para compreender:

- Como os dados são lidos do ficheiro CSV
- Como as características e labels são separadas
- Como os dados são convertidos para arrays NumPy
- Como a função lida com diferentes formatos de dados

3.5 2.d) Retorno ao simple_dataset

Alteramos o mecanismo de carregamento de dados para utilizar novamente `simple_dataset`, que fornece um conjunto de dados pequeno e controlado ideal para testes e compreensão inicial dos métodos de divisão.

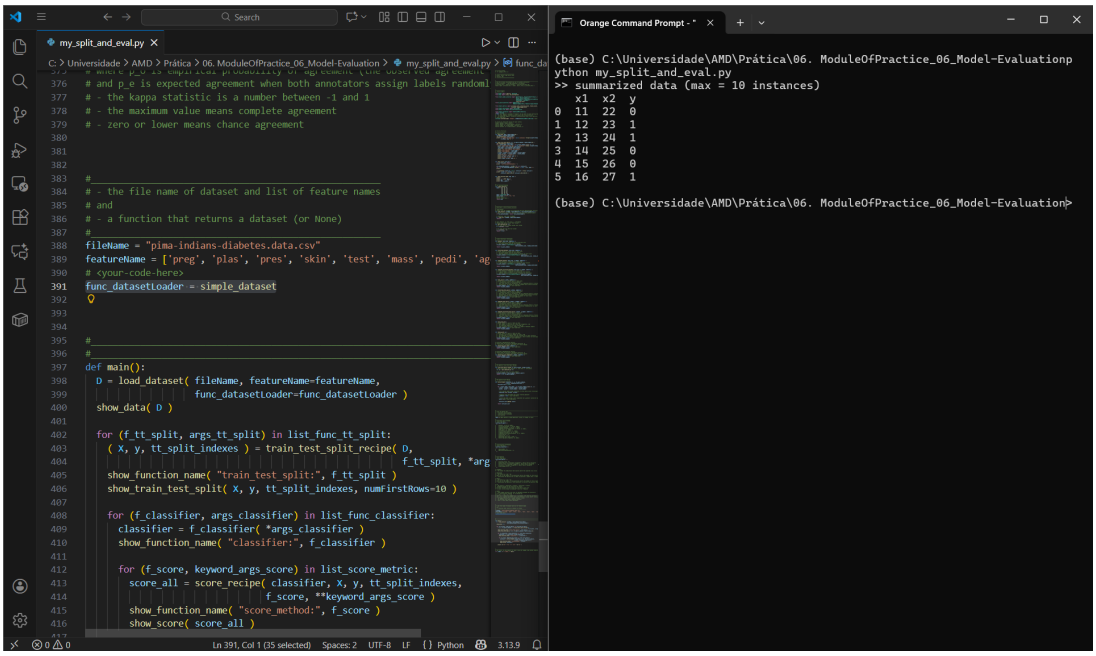


Figura 4: Carregamento do dataset simples através da função simple_dataset

4 Aplicação de Métodos de Separação Train-Test

4.1 Objetivo

Aplicar e compreender diferentes técnicas de separação de dados em conjuntos de treino e teste, essenciais para a avaliação adequada de modelos de machine learning.

4.2 3.a) Método Holdout

Aplicamos o método holdout ao dataset simples fornecido pelo script. Este método divide os dados numa única vez em conjuntos de treino e teste.

Configuração padrão:

- Proporção de teste: 1/3 (aproximadamente 33%)
- Proporção de treino: 2/3 (aproximadamente 67%)
- Seed aleatório para reprodutibilidade

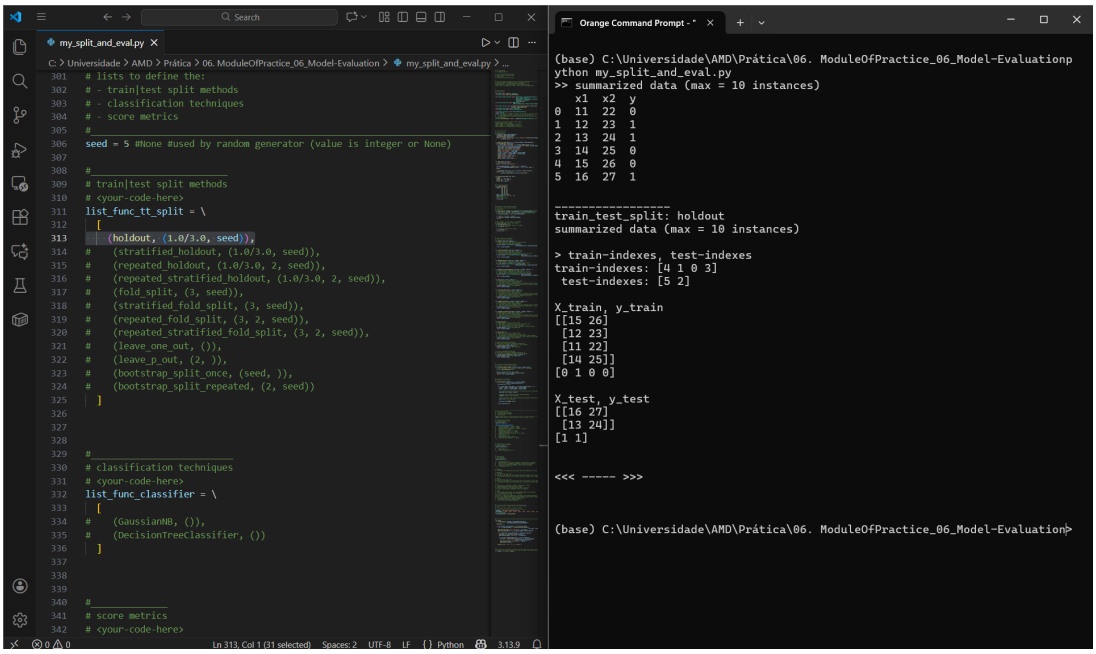


Figura 5: Aplicação do método holdout ao dataset simples

4.3 3.b) Método Holdout com Divisão 50/50

Repetimos o método holdout, alterando o rácio para 50% train e 50% test. Esta configuração:

- Equilibra o tamanho dos conjuntos de treino e teste
- Pode ser útil quando se tem dados limitados
- Reduz o tamanho do conjunto de treino, potencialmente afetando o desempenho do modelo

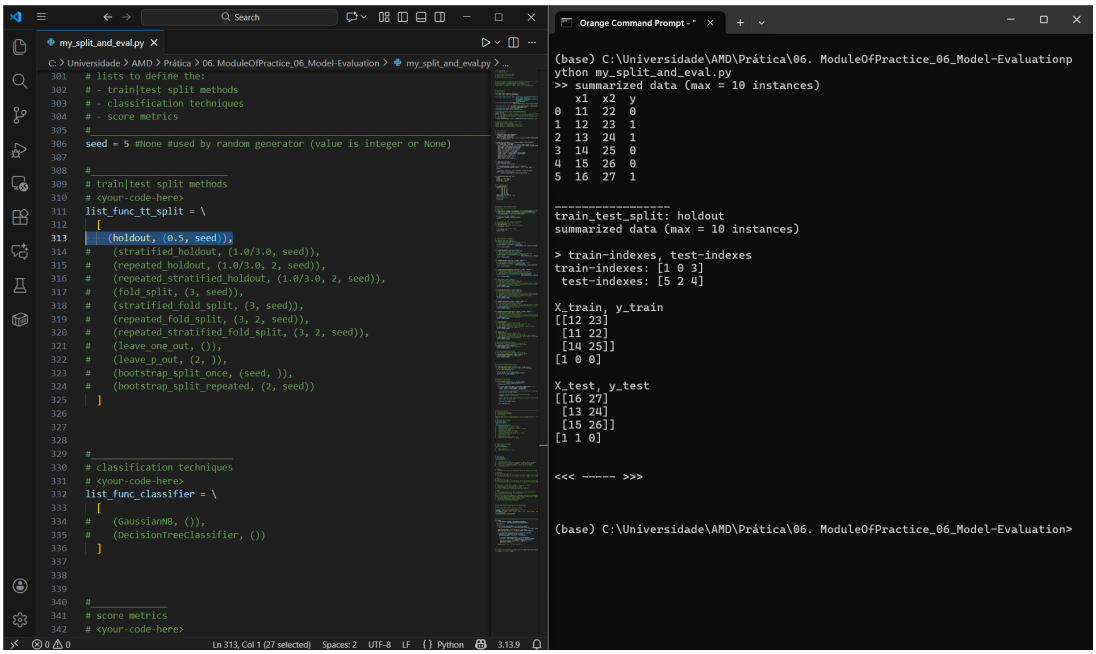


Figura 6: Método holdout com divisão 50/50 aplicado ao dataset simples

4.4 3.c) Método Stratified Holdout

Aplicamos o método `stratified_holdout`, que preserva a proporção das classes no processo de divisão train-test.

Vantagens da divisão estratificada:

- Mantém a distribuição de classes em ambos os conjuntos
- Particularmente importante para datasets desequilibrados
- Reduz a variância nas estimativas de desempenho
- Garante que ambos os conjuntos são representativos da distribuição original

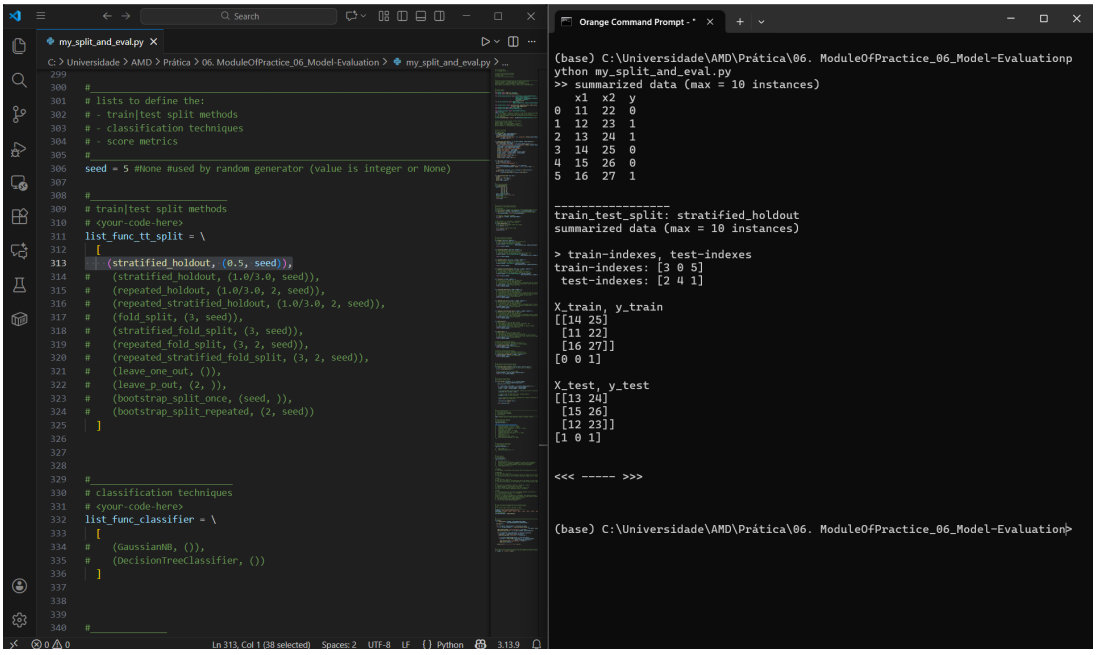


Figura 7: Divisão estratificada 50/50 preservando as proporções das classes

4.5 3.d) Método Repeated Stratified Holdout

Aplicamos o método `repeated_stratified_holdout`, que executa várias divisões estratificadas sucessivas.

Configuração utilizada:

- Divisão 50/50
- 2 repetições com seeds diferentes
- Preservação da proporção de classes em cada repetição

Benefícios das repetições:

- Reduz a dependência de uma única divisão aleatória
- Fornece múltiplas estimativas de desempenho

- Permite calcular média e desvio padrão das métricas
- Aumenta a confiança estatística nos resultados

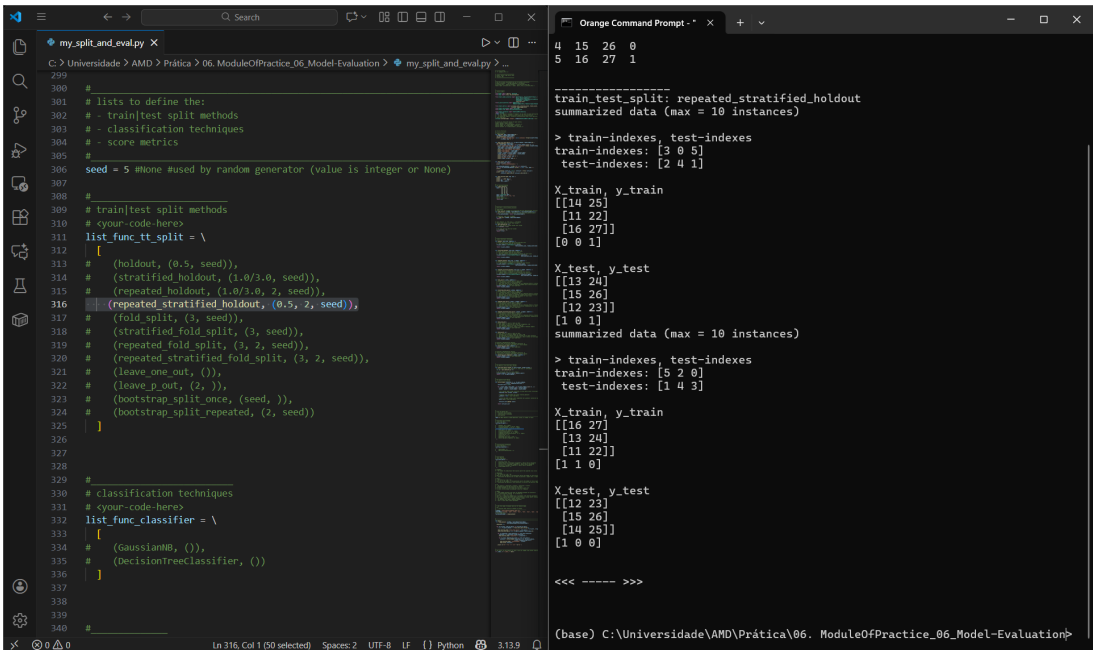


Figura 8: Aplicação do método repeated_stratified_holdout com duas repetições

5 Código Completo de Train-Test e Análise de Resultados

5.1 Objetivo

Aprofundar as técnicas de validação, implementando vários métodos e analisando o seu comportamento em diferentes cenários.

5.2 4.a) Implementação de Métodos Adicionais

Todos os métodos restantes foram implementados no ficheiro `my_split_and_eval.py`. Estes métodos incluem:

- **leave_one_out:** Remove uma instância de cada vez para teste
- **leave_p_out:** Remove p instâncias de cada vez para teste
- **k_fold:** Divide os dados em k partições
- **stratified_k_fold:** K-fold com preservação de proporções de classes
- **bootstrap:** Amostragem com reposição

5.3 4.b) Análise de Estratégias

Os métodos implementados cobrem um vasto leque de estratégias – desde abordagens exaustivas, como `leave_one_out` e `leave_p_out`, até técnicas que criam múltiplas divisões com menor custo computacional, como `k_fold` e `bootstrap`.

Comparação de métodos:

Método	Custo Computacional	Variância	Uso Recomendado
Holdout	Muito baixo	Alta	Datasets grandes
Leave-one-out	Muito alto	Muito baixa	Datasets pequenos
K-fold	Moderado	Moderada	Uso geral
Bootstrap	Configurável	Baixa	Datasets médios

Tabela 1: Comparação de métodos de validação

Cada método serve um propósito diferente, sendo importante considerar qual é o mais adequado para a situação específica.

5.4 4.c) Resultados e Incompatibilidades

As funções produziram resultados de classificação variados: algumas entregaram divisões precisas, enquanto outras não tiveram um desempenho tão bom.

Problemas identificados:

- Certas abordagens mostraram-se incompatíveis com o dataset utilizado
- Os métodos `leave_one_out` e `leave_p_out` entraram em loop infinito com o dataset pequeno
- A incompatibilidade impediu a obtenção de resultados de classificação para estes métodos

Lições aprendidas:

- A escolha do método de validação deve considerar o tamanho do dataset
- Métodos exaustivos podem ser impraticáveis para certos datasets
- É importante testar a viabilidade computacional antes da aplicação em produção

6 Análise e Implementação Bootstrap

6.1 Objetivo

Analisar o método bootstrap e corrigir a implementação incompleta fornecida, compreendendo os princípios da amostragem com reposição.

6.2 5.a) Execução do Script Bootstrap

Executamos o ficheiro `my_split_bootstrap.py` utilizando o VSCode. O script inicial apresentava uma implementação incompleta do método de divisão bootstrap.

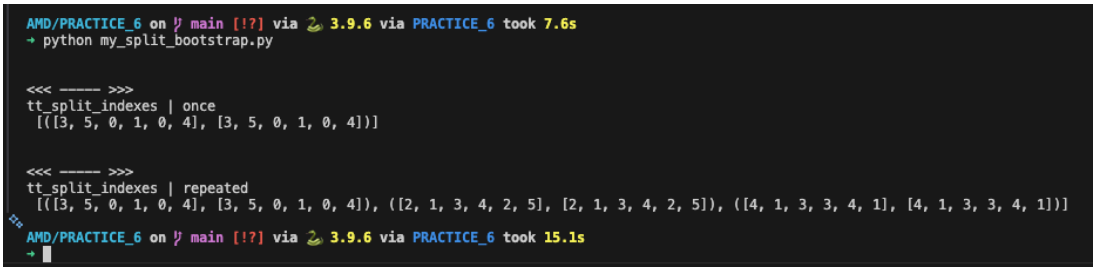


Figura 9: Output da execução de my_split_bootstrap.py

6.3 5.b) Correção do Conjunto de Teste

Identificamos que o conjunto de teste deve conter apenas amostras não utilizadas no treino. Implementamos um filtro dos índices originais para manter apenas aqueles ausentes do conjunto de treino, permitindo uma avaliação adequada do modelo.

Implementação correta:

```
# Filtrar indices para obter apenas amostras nao utilizadas no
    treino
test_indices = [i for i in range(len(X)) if i not in
    train_indices]
```

Justificação:

- Garante independência entre conjuntos de treino e teste
- Evita sobreajuste artificial nas métricas de avaliação
- Mantém a integridade da validação do modelo

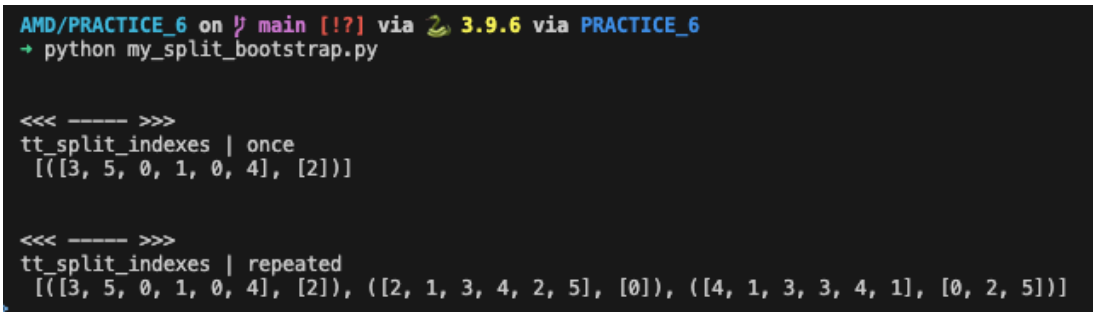


Figura 10: Output após atualização do método de divisão

6.4 5.c) Teste com Diferentes Repetições

Testamos a reamostragem bootstrap com 10, 50 e 100 repetições para demonstrar como o aumento do número de divisões treino-teste fornece uma avaliação de modelo mais robusta através de múltiplas rondas independentes de validação.

Observações:

Repetições	Tempo de Execução	Estabilidade
10	Rápido	Moderada
50	Moderado	Boa
100	Mais lento	Muito boa

Tabela 2: Impacto do número de repetições bootstrap

- Maior número de repetições aumenta a confiança estatística
- Existe um compromisso entre precisão e custo computacional
- Para datasets médios, 50-100 repetições são geralmente suficientes

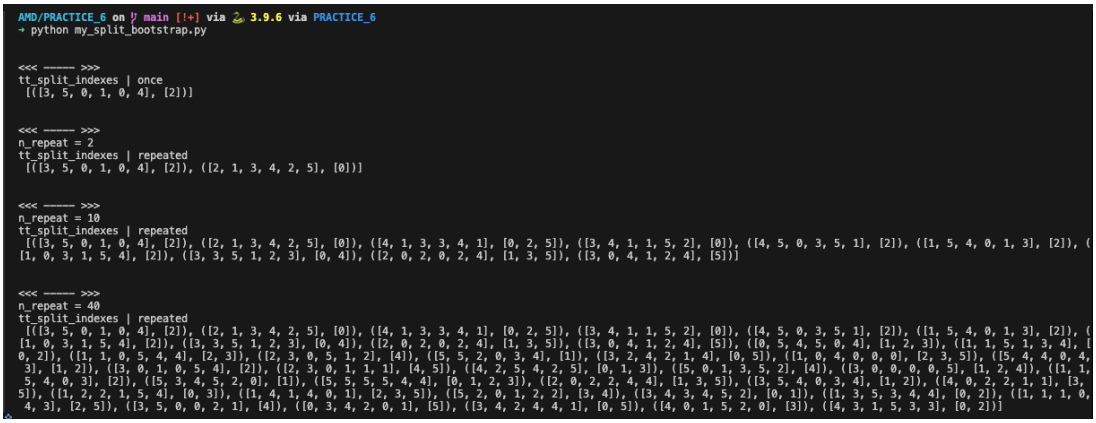


Figura 11: Output com diferentes números de repetições (10, 50, 100)

6.5 5.d) Integração no Script Principal

Implementamos métodos de divisão treino-teste bootstrap criando instâncias das classes `MyBootstrapSplitOnce` e `MyBootstrapSplitRepeated`, já importadas no início do ficheiro, permitindo reamostragem bootstrap simples e repetida para avaliação de modelos.

Integração no `my_split_and_eval.py`:

```
# Bootstrap simples
("bootstrap_once", MyBootstrapSplitOnce(), ()),

# Bootstrap repetido
("bootstrap_repeated", MyBootstrapSplitRepeated(n_repeats=50), ()
    ),
```

7 Construir um Classificador com Divisão Treino-Teste

7.1 Exercício 6 (b): Aplicar o DecisionTreeClassifier

O exercício solicita procurar a tag `<your-code-here>` e aplicar o `DecisionTreeClassifier`. Ao executar o código inicial, observámos que "nada aconteceu"porque, embora o classifi-

cador tenha sido criado, nenhum método de divisão treino-teste estava ativo para treinar e avaliar o modelo.

7.1.1 Implementação

Na lista de classificadores (linhas 354-358), ativamos o `DecisionTreeClassifier`:

```
list_func_classifier = \
  [
#    (GaussianNB, ()),
   (DecisionTreeClassifier, ())
  ]
```

Esta configuração é utilizada na função `main()` (linha 409) onde o classificador é instanciado:

```
classifier = f_classifier( *args_classifier )
```

Que resulta em `classifier = DecisionTreeClassifier()`.

Sem um método de divisão treino-teste ativo na lista `list_func_tt_split`, o classificador não é treinado nem avaliado, resultando em nenhuma saída visível no terminal.

7.2 Exercício 6 (c): Aplicar a Divisão Holdout

O próximo passo foi ativar o método de divisão treino-teste `holdout` (utilizando `DecisionTreeClassifie`
Ao executar o código, observámos que agora é necessário aplicar a função `score_recipe` utilizando algum(ns) método(s) de pontuação para avaliar o desempenho do modelo.

7.2.1 Implementação

Na lista de métodos de divisão (linhas 328-341), ativamos o método `holdout`:

```
list_func_tt_split = \
  [
   (holdout, (1.0/3.0, seed)),
#    (stratified_holdout, (1.0/3.0, seed)),
#    (repeated_holdout, (1.0/3.0, 2, seed)),
#    ...
  ]
```

Isto configura uma divisão holdout com 1/3 dos dados para teste (aproximadamente 33%) e 2/3 para treino (aproximadamente 67%), utilizando `seed=5` para reprodutibilidade.

7.2.2 Método de Divisão Holdout

O método holdout é a abordagem mais simples para divisão treino-teste. Divide o conjunto de dados numa única vez em dois subconjuntos:

- **Conjunto de Treino:** Utilizado para treinar o modelo (tipicamente 60-80% dos dados)
- **Conjunto de Teste:** Utilizado para avaliar o desempenho do modelo (tipicamente 20-40% dos dados)

Vantagens do Holdout:

- Computacionalmente eficiente (apenas uma divisão)
- Simples de implementar e compreender
- Adequado para conjuntos de dados grandes

Desvantagens do Holdout:

- Alta variância nos resultados (dependente da divisão específica)
- Pode não ser representativo para conjuntos de dados pequenos
- Utiliza menos dados para treino em comparação com validação cruzada

7.2.3 Implementação

No código fornecido, o método holdout está configurado como:

```
( "holdout", holdout, (1.0/3.0, 5) ),
```

Isto configura:

- **Proporção de teste:** 1/3 (aproximadamente 33%) dos dados para teste
- **Proporção de treino:** 2/3 (aproximadamente 67%) dos dados para treino
- **Seed aleatório:** 5 (para reprodutibilidade)

7.2.4 Classificador: Árvore de Decisão

O exercício utiliza o `DecisionTreeClassifier` do scikit-learn. As árvores de decisão são uma escolha popular devido à sua:

- Interpretabilidade (estrutura em árvore visualizável)
- Capacidade de lidar com características numéricas e categóricas
- Não necessidade de normalização de características
- Capacidade de capturar relações não-lineares

7.2.5 Conjunto de Dados: Pima Indians Diabetes

O conjunto de dados utilizado contém:

- **Instâncias:** 768 pacientes
- **Características:** 8 características médicas (gravidez, glucose, pressão arterial, etc.)
- **Classes:** 2 (diabético vs. não-diabético)
- **Distribuição:** 268 positivos (34.9%), 500 negativos (65.1%)

7.2.6 Resultados

Ao executar o script com apenas o método holdout ativo, obtivemos:

Métrica	Valor
Exatidão (Accuracy)	66.15%

Tabela 3: Resultados do Exercício 6 utilizando divisão holdout

Interpretação dos Resultados:

- A exatidão de 66.15% significa que o classificador prevê corretamente a classe para aproximadamente 2 em cada 3 instâncias
- Este valor está acima do baseline de classe maioritária (65.1%), indicando que o modelo aprendeu alguns padrões
- No entanto, há espaço significativo para melhoria, possivelmente através de:
  - Ajuste de hiperparâmetros da árvore de decisão
  - Engenharia de características
  - Tentativa de diferentes algoritmos de classificação
  - Tratamento de valores em falta ou outliers

8 Múltiplas Métricas e Divisões Repetidas

8.1 (d) Completar a Função score_recipe

Procurámos a tag `<your-code-here>` na função `score_recipe` (linhas 296-314) e completámos o código:

```
# predict using the model and X_test (testing dataset)
y_predict = classifier.predict( X_test )

# score the model using y_test (expected) and y_predict (
   predicted by model)
score = f_score( y_test, y_predict, **keyword_args_score )
```

8.2 (e) Testar o Código

Testámos o código com a configuração especificada: `repeated_holdout(1.0/3.0, 2, seed)` e `DecisionTreeClassifier` com `seed=5`.

Resultados obtidos:

```
score_method: accuracy_score
::all-evaluated-datasets::
50.00% | 0.00% |
25.00% (+/- 25.00%)

score_method: precision_score
::all-evaluated-datasets::
100.00% | 0.00% |
50.00% (+/- 50.00%)

score_method: recall_score
::all-evaluated-datasets::
50.00% | 0.00% |
25.00% (+/- 25.00%)

score_method: f1_score
::all-evaluated-datasets::
66.67% | 0.00% |
33.33% (+/- 33.33%)

score_method: cohen_kappa_score
::all-evaluated-datasets::
0.00% | 0.00% |
0.00% (+/- 0.00%)
```

8.3 (f) Observar os Avisos

Durante a execução, surgiram avisos (warnings). Identificámos os seguintes textos importantes:

Aviso "no predicted samples":

```
UndefinedMetricWarning: Precision is ill-defined and being set to
    0.0
due to no predicted samples.
```

Aviso "no true samples":

```
UndefinedMetricWarning: Recall is ill-defined and being set to
   0.0
due to no true samples.
```

8.4 (g) Explicar a Diferença entre Avisos

8.4.1 Diferença entre Avisos

"no predicted samples":

- Significa que o classificador não previu nenhuma instância como positiva (TP + FP = 0)
- Todas as previsões foram da classe negativa

- Afeta a métrica de precisão (precision)

"no true samples":

- Significa que o conjunto de teste não contém nenhuma instância positiva verdadeira (TP + FN = 0)
- Todas as instâncias reais no conjunto de teste são da classe negativa
- Afeta a métrica de revocação (recall)

8.4.2 Aviso: no true samples (recall e f1-score)

A revocação (recall) é definida como:

$$\text{Recall} = \frac{TP}{TP + FN} \tag{1}$$

Quando não há amostras verdadeiras positivas no conjunto de teste, temos TP + FN = 0, o que resulta em divisão por zero. O F1-score depende de recall, portanto herda este problema.

8.4.3 Aviso: no predicted samples (precision e f1-score)

A precisão (precision) é definida como:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{2}$$

Quando o classificador não prevê nenhuma amostra como positiva, temos TP + FP = 0, o que resulta em divisão por zero. O F1-score depende de precision, portanto herda este problema.

8.5 (h) Utilizar o Parâmetro zero_division

Lemos a documentação do scikit-learn para as funções `precision_score` e `recall_score`. Utilizámos o parâmetro `zero_division` para eliminar os avisos.

Na lista de métricas (linhas 365-373):

```
list_score_metric = \
  [
   (accuracy_score, {}),
   (precision_score, {"average":"weighted", "zero_division":0}),
   (recall_score, {"average":"weighted", "zero_division":0}),
   (f1_score, {"average":"weighted", "zero_division":0}),
   (cohen_kappa_score, {}),
  ]
```

O parâmetro `zero_division=0` define o valor da métrica como 0.0 quando ocorre divisão por zero, eliminando os avisos.

8.6 (i) Desativar Avisos UndefinedMetricWarning

Procurámos a tag `<your-code-here>` e utilizámos o valor "ignore"para desativar a apresentação de avisos da categoria `UndefinedMetricWarning`.

No início do ficheiro (linhas 39-43):

```
1 from sklearn.exceptions import UndefinedMetricWarning
2 import warnings
3
4 warnings.filterwarnings( "ignore", category=
    UndefinedMetricWarning )
```

Aviso importante: Ao desativar estes avisos, perde-se informação importante sobre problemas no conjunto de dados ou no modelo. Só se deve fazer isto quando realmente se conhece o impacto correspondente!

9 Conclusão

A realização deste laboratório permitiu percorrer todas as etapas de avaliação de modelos preditivos: preparação do ambiente, carregamento de dados, divisão correta de test-train, implementação de métodos avançados de amostragem e integração de métricas de desempenho.

9.1 Competências Adquiridas

Ao longo dos sete exercícios, desenvolvemos competências em:

1. **Preparação de Ambiente:** Integração de bibliotecas Python (pandas, numpy, scikit-learn) no ambiente Orange Data Mining
2. **Gestão de Dados:** Carregamento e processamento de datasets reais (Pima Indians Diabetes) e sintéticos
3. **Métodos de Validação:** Implementação e comparação de múltiplos métodos de divisão treino-teste:
   - Holdout simples e estratificado
   - Holdout repetido
   - Leave-one-out e leave-p-out
   - K-fold e stratified k-fold
   - Bootstrap simples e repetido
4. **Construção de Classificadores:** Implementação de árvores de decisão com scikit-learn
5. **Avaliação de Desempenho:** Aplicação de múltiplas métricas (exatidão, precisão, revocação, F1-score, Kappa de Cohen)
6. **Análise Crítica:** Interpretação de avisos, identificação de limitações e gestão de casos extremos

9.2 Principais Aprendizagens

- **Tamanho do dataset importa:** Conjuntos de dados pequenos levam a métricas não fiáveis e avisos frequentes
- **Múltiplas métricas são essenciais:** Exatidão por si só pode ser enganadora; precisão, revocação e F1-score fornecem visão completa
- **Escolha adequada do método:** Cada técnica de validação tem o seu caso de uso ideal, considerando tamanho do dataset e custo computacional
- **Estratificação é crucial:** Preservar proporções de classes melhora significativamente a qualidade da validação
- **Repetições aumentam robustez:** Múltiplas divisões fornecem estimativas mais estáveis e confiáveis
- **Bootstrap é versátil:** A amostragem com reposição oferece um bom compromisso entre precisão e eficiência
- **Tratamento de avisos:** Compreender os avisos de métricas indica problemas de dados reais, não apenas questões técnicas

9.3 Visão Final

No final, o laboratório fornece uma visão completa das práticas recomendadas para avaliar modelos de Machine Learning de forma rigorosa. A experiência demonstra que a avaliação adequada de classificadores requer:

- Conjuntos de dados de tamanho suficiente para validação robusta
- Múltiplas perspetivas através de diferentes métricas
- Compreensão profunda das limitações de cada método e métrica
- Tratamento cuidadoso de casos extremos e avisos
- Múltiplas avaliações para estimativas estáveis e confiáveis
- Escolha informada do método de validação baseada no contexto do problema

Estes exercícios estabelecem uma base sólida para técnicas de avaliação mais avançadas e para a aplicação prática de machine learning em problemas reais, garantindo que os modelos desenvolvidos são devidamente validados e têm desempenho confiável em dados não vistos.